

SMART EMERGENCY PATIENT COORDINATION SYSTEM

A CONCURRENCY-SAFE EMERGENCY PATIENT MANAGEMENT PLATFORM

PROJECT FOCUS Race-condition-safe patient updates using POSIX threads, patient-level mutex protection, emergency priority IPC, a separate Treatment Module and audit logging.

Technical Project Report
C Programming • Operating Systems • Concurrency

Executive Snapshot

The Smart Emergency Patient Coordination System is designed around a central systems problem: multiple hospital departments may need to access or update the same emergency patient record at the same time. The system aims to preserve patient-record correctness while retaining useful concurrency. It combines a central patient registry, POSIX threads, patient-level mutex synchronization, emergency priority handling, a separate Treatment Module using process separation, POSIX message queues for inter-process communication, and file-based audit logging.

Dimension	Proposed Approach
Concurrency	POSIX threads with patient-level mutex protection
Correctness	Protected read–validate–update critical section
Emergency Communication	Priority-based POSIX message queue
Process Separation	Separate Treatment Module using fork()
Persistence / Audit	File-based patient activity history
Memory	Dynamic patient registry using malloc()/realloc()
User Interface	Web prototype for registration, search, update, treatment, discharge and audit

01 Project Overview & Problem Definition

Abstract

The proposed system models an emergency-care environment in which several departments can work on patient information concurrently. The engineering challenge is to keep shared patient state consistent without unnecessarily serializing unrelated work. The design therefore protects each patient record independently, gives emergency requests explicit priority, separates treatment processing from the main registration workflow, and records important lifecycle activities for traceability.

1. The Shared Patient Record Race Condition

A patient record is a shared mutable resource. An update may involve reading the current state, validating a change, modifying the record, and recording the operation. If two department threads perform this sequence without synchronization, both may observe the same stale state and then write conflicting results.

CORE FAILURE Concurrent read–modify–write operations can interleave, causing one department's update to overwrite another department's change.

Impact

- Two departments can observe the same pre-update patient state.
- A validation decision can become invalid before the update is committed.
- The final patient record may not represent the intended sequence of operations.
- The exact incorrect state can depend on thread scheduling and timing.
- Critical emergency requests require priority-based processing rather than ordinary ordering.

02 Objectives, Architecture & Technology Stack

2. Project Objectives

- Implement concurrent patient processing using POSIX threads.
- Protect patient-level read–modify–update operations using `pthread_mutex_t`.
- Allow operations on different patients to proceed concurrently.
- Handle Level 1, Level 2 and Level 3 emergency requests using priority-based processing.
- Use `fork()` to separate the Treatment Module from the main processing flow.
- Use POSIX message queues for prioritized inter-process communication.
- Maintain an audit history for registration, update, treatment and discharge operations.
- Support a web prototype that demonstrates the complete patient workflow.

3. Architecture

Layer	Components	Responsibility
Client / UI	Web prototype	Issue patient operations and visualize status
Patient Registry	Dynamic patient records	Store and retrieve current patient state
Concurrency	POSIX threads + mutex	Concurrent department work and safe updates
Emergency Control	Level 1/2/3 priority	Classify and prioritize urgent cases
IPC	POSIX message queue	Deliver emergency treatment requests
Treatment	<code>fork()</code> Treatment Module	Process emergency treatment requests separately
Audit	History / audit log	Record important patient lifecycle activities

4. Technology Stack

Area	Technology	Role
Language	C / GNU11	Core implementation
Threads	POSIX pthreads	Concurrent departmental workers
Synchronization	<code>pthread_mutex_t</code>	Protect shared patient state
Process	<code>fork()</code>	Separate Treatment Module
IPC	POSIX message queue	Prioritized inter-process communication
Memory	<code>malloc()</code> , <code>realloc()</code>	Dynamic patient registry
Web	HTML, CSS, JavaScript	Interactive prototype
Audit	File logging	Traceable operation history
Environment	Linux / GCC	Build and execution

DESIGN PRINCIPLE Do not serialize the entire hospital workflow. Protect only the patient record that is currently

being modified so unrelated patients can continue concurrently.

03 Concurrency Control & Safe Patient Updates

5. Patient-Level Mutex Synchronization

The system uses a mutex associated with each patient record. A department thread acquires the patient mutex before reading the shared state and keeps it through validation and update. The lock is released only after the protected operation is complete.

CRITICAL SECTION pthread_mutex_lock(&patient->mutex) → Read → Validate → Update → Record Operation
→ pthread_mutex_unlock(&patient->mutex)

Without Synchronization	With Patient-Level Mutex
Two departments may read the same patient state.	Only one thread enters the critical section for that patient at a time.
Validation may use stale state.	Validation uses the protected current state.
Updates may overwrite each other.	Same-patient updates are serialized safely.
Correctness depends on timing.	Protected correctness is independent of thread interleaving.

6. Why Fine-Grained Locking?

A single global system mutex would force unrelated patient operations to wait. Patient-level locking provides a better concurrency trade-off: operations that compete for the same patient are serialized, while operations involving different patients can continue in parallel.

Concurrency Example

Doctor Thread → P1001 Mutex → Read/Validate/Update → Unlock
Nurse Thread → waits for P1001 mutex → reads the updated state → performs the next safe update.
At the same time, a thread working on P1002 can proceed using P1002's independent mutex.

04 Emergency Priority, IPC & Treatment Module

7. Critical Conditions

Emergency Level	Condition	System Response
Level 1	Critical / life-threatening	Highest priority and immediate treatment dispatch
Level 2	Serious emergency	High-priority processing
Level 3	Less critical	Normal emergency processing

Priority handling ensures that emergency treatment requests are not treated as ordinary patient operations. The most critical request is placed ahead of lower-severity requests in the treatment communication path.

8. POSIX Message Queue & Treatment Module

When an emergency request is generated, the main system classifies the emergency level and sends the request through a POSIX message queue. The Treatment Module runs as a separate process created with `fork()`. This separates treatment handling from the main patient-registration workflow.

Stage	Action
1	Emergency condition is detected.
2	Patient is assigned Level 1, Level 2 or Level 3.
3	Treatment request is placed into the prioritized message queue.
4	Separate Treatment Module receives the request.
5	Urgent requests are handled according to priority.
6	Treatment/discharge activity is recorded in the audit history.

9. Prototype Interface

The web prototype demonstrates Login, Dashboard, Patient Registration, Patient Search, Patient Update, Emergency Treatment, Patient Discharge and Audit Log screens. Sample patient data is used to make the workflow demonstrable without requiring a production hospital database.

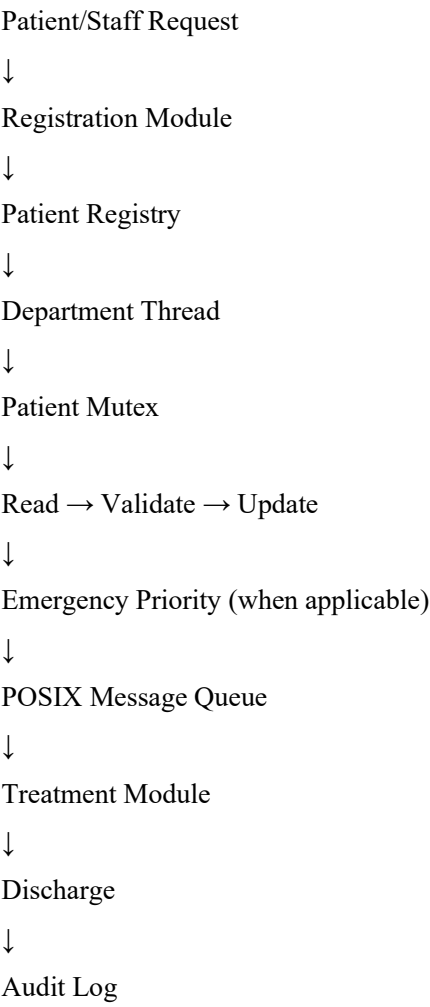
05 Auditability & End-to-End Patient Lifecycle

10. Audit Logging

The audit log records important patient lifecycle activities in an ordered history. This provides traceability for registration, search, update, emergency treatment and discharge operations. The audit history is separate from the current in-memory patient state.

Operation	Example Record
REGISTER	P1001 registered in Emergency Department
SEARCH	P1001 details viewed
UPDATE	P1001 status/department updated
TREATMENT	P1001 Level 1 request dispatched
DISCHARGE	P1001 treatment completed and discharged

11. End-to-End Patient Lifecycle



TRACEABILITY PRINCIPLE Every major patient operation should be safe, visible and traceable through the

system lifecycle.

06 Testing, Expected Results, Limitations & Conclusion

12. Validation Strategy

Test Scenario	Expected Safe Behavior	Purpose
Same-Patient Concurrency	Updates to one patient are serialized.	Verify mutex protection
Different-Patient Concurrency	Different patients can be processed concurrently.	Verify useful parallelism
Level 1 Emergency	Critical request receives highest priority.	Verify emergency prioritization
Treatment IPC	Request reaches separate Treatment Module.	Verify inter-process communication
Audit Verification	Lifecycle operations appear in history.	Verify traceability
Patient Lifecycle	Register → Search → Update → Treat → Discharge works.	Verify end-to-end workflow

13. Key Outcomes

- Patient-level synchronization protects shared patient state.
- Different patient records can be processed concurrently.
- Critical emergency requests receive priority treatment handling.
- Treatment processing is separated from the main registration process.
- Audit logging creates a traceable patient activity history.
- The web prototype makes the system workflow easy to demonstrate.

14. Limitations & Future Enhancements

- This is a prototype and is not a production hospital information system.
- Production deployment would require strong authentication and authorization.
- A real hospital database and robust persistence guarantees would be required.
- Encrypted communication such as TLS can be added for networked deployment.
- Future versions can add role-based access, real-time dashboards, metrics and distributed tracing.
- Additional fault-injection and stress testing would be needed before production use.

15. Conclusion

The Smart Emergency Patient Coordination System demonstrates how operating-system concepts can be applied to emergency patient coordination. The central lesson is that useful concurrency and correctness can coexist when shared patient state is protected at the appropriate granularity. Patient-level mutexes protect critical updates, priority IPC supports urgent treatment requests, process separation isolates treatment handling, and audit logging provides traceability across the patient lifecycle.

IN ONE SENTENCE The system combines safe concurrent patient-record management, priority emergency
--

communication, process separation and auditable history into one demonstrable platform.

Keywords: C • GNU11 • POSIX Threads • pthread_mutex_t • fork() • POSIX Message Queue • Race Condition • Patient Registry •
Emergency Priority • Audit Logging